Rapport de Projet

Implémentation d'algorithmes de flots Flot maximum, coupe minimale et flot de coût minimum (Java)

Réalisé par
NOM Prénom

Encadré par
Encadrant(e) / enseignant(e)

Master Informatique
à Université Côte d'Azur
Avril 2026

Résumé

Ce rapport présente une implémentation *from scratch* en Java (sans bibliothèques de flots) de trois briques classiques de recherche opérationnelle sur réseaux : (i) le calcul d'un flot maximum par recherche de chemins augmentants dans le graphe résiduel (Edmonds–Karp), (ii) l'extraction d'une coupe minimale (min-cut) à partir du graphe résiduel final, (iii) le calcul d'un flot de coût minimum par augmentations successives (Successive Shortest Path) en deux variantes : Bellman–Ford (coûts négatifs) et Dijkstra avec renormalisation par potentiels. Nous décrivons la modélisation, la structure de données du graphe résiduel, les algorithmes, une analyse de complexité, et une stratégie de validation (benchmarks de cohérence entre les deux variantes de flot de coût minimum).

Table des matières

1 Introduction **3**
1.1 Objectifs et livrables . . . 3
1.2 Organisation . . . 3

2 Modélisation : graphes, flots, résiduel **3**
2.1 Graphe et flot . . . 3
2.2 Graphe résiduel . . . 3

3 Structure de données **3**
3.1 Listes d'adjacence et arcs couplés . . . 3
3.2 Convention d'indexation . . . 4

4 Algorithmes **4**
4.1 Flot maximum : Edmonds–Karp . . . 4
4.2 Coupe minimale (min-cut) . . . 4
4.3 Flot de coût minimum : Successive Shortest Path . . . 4
4.4 Détection de cycles négatifs . . . 4

5 Complexité **5**

6 Validation et résultats **5**
6.1 Jeux de test . . . 5
6.2 Bench de cohérence . . . 5
6.3 Exemple de sortie (résumé) . . . 5

7 Conclusion **5**

1 Introduction

1.1 Objectifs et livrables

La consigne demande (i) l'implémentation d'un algorithme de flot maximum (Ford–Fulkerson) avec affichage du flot sur chaque arc, (ii) l'identification et l'extraction d'une coupe minimale (min-cut), (iii) un module de flot de coût minimum avec deux variantes de plus court chemin, et (iv) une détection de cycles négatifs dans le graphe résiduel pour garantir l'optimalité.

Le projet est réalisé en Java et ne dépend que de la bibliothèque standard (aucune librairie de flots, NetworkX/igraph, etc.).

1.2 Organisation

Le rapport est organisé comme suit : modélisation et graphe résiduel (Section 2), structure de données (Section 3), algorithmes (Section 4), complexités (Section 5), validation (Section 6).

2 Modélisation : graphes, flots, résiduel

2.1 Graphe et flot

On considère un graphe orienté $G = (X, U)$, où chaque arc $(i, j) \in U$ possède une capacité $u(i, j)$ et (èventuellement) un coût $c(i, j)$. Un flot est une fonction $f(i, j)$ telle que :

— **Capacité** : $0 \leq f(i, j) \leq u(i, j)$.

— **Conservation** : pour tout noeud $v \neq s, t$, $\sum_i f(i, v) = \sum_j f(v, j)$.

2.2 Graphe résiduel

Le graphe résiduel $R(f)$ est essentiel pour Ford–Fulkerson et pour le flot de coût minimum. Pour chaque arc (i, j) du graphe initial :

— si $f(i, j) < u(i, j)$, on ajoute un **arc avant** (i, j) de capacité résiduelle $r(i, j) = u(i, j) - f(i, j)$;

— si $f(i, j) > 0$, on ajoute un **arc arrière** (j, i) de capacité résiduelle $f(i, j)$.

Dans le cas du flot de coût minimum, l'arc arrière doit avoir un coût opposé : $c(j, i) = -c(i, j)$.

3 Structure de données

3.1 Listes d'adjacence et arcs couplés

Le graphe résiduel est représenté par des listes d'adjacence. Chaque arc original $u \to v$ est encodé par une paire d'arêtes couplées :

— une arête « avant » de capacité `cap` et coût `cost`,

— une arête « arrière » de capacité initiale 0 et coût $-$`cost`.

Les mises à jour lors d'une augmentation modifient simultanément la capacité résiduelle de l'arête avant et de l'arête arrière.

3.2 Convention d'indexation

En interne, on utilise systématiquement les indices $0..n-1$ (adapté aux tableaux Java). Pour les instances fournies avec des sommets $1..n$, un flag explicite `-one-based` convertit les sommets lus via -1. Les sorties réutilisent la convention choisie (0-based par défaut, 1-based avec `-one-based`).

4 Algorithmes

4.1 Flot maximum : Edmonds–Karp

La recherche de chemins augmentants est effectuée dans $R(f)$ par BFS (Edmonds–Karp). À chaque itération :

1. on trouve un chemin $s \to t$ sur les arêtes de capacité résiduelle strictement positive,
2. on calcule le goulot $\Delta = \min r(i,j)$ le long du chemin,
3. on augmente le flot (mise à jour des capacités résiduelles avant/arrière).

4.2 Coupe minimale (min-cut)

Une fois le flot maximum atteint, on calcule S, l'ensemble des sommets atteignables depuis s dans le graphe résiduel final. La coupe minimale est donnée par les arcs originaux (i,j) tels que $i \in S$ et $j \notin S$.

4.3 Flot de coût minimum : Successive Shortest Path

On cherche à envoyer du flot par augmentations successives en suivant à chaque fois un chemin de coût minimal dans le graphe résiduel. Cette approche permet :

— un **mode F fixé** : s'arrêter à l'atteinte d'une demande F (si F est fourni),

— un **mode saturation** : pousser jusqu'à impossibilité d'augmenter (min-cost max-flow) si F est absent.

Variante A : Bellman–Ford. Bellman–Ford est utilisé à chaque augmentation pour gérer les coûts négatifs (liés aux arcs arrière).

Variante B : Dijkstra + renormalisation. Pour permettre Dijkstra, on utilise des potentiels π et les coûts réduits :

$$c_R(u,v) = \pi[u] + c(u,v) - \pi[v] \geq 0.$$

Dijkstra est exécuté sur c_R (PriorityQueue), puis les potentiels sont mis à jour.

4.4 Détection de cycles négatifs

Un flot est de coût minimum si et seulement si le graphe résiduel ne contient aucun cycle de coût négatif. On détecte un tel cycle en relaxant toutes les arêtes résiduelles (capacité > 0) n fois (Bellman–Ford). Si une amélioration est encore possible à la n-ième passe, un cycle négatif existe.

5 Complexité

— Edmonds–Karp : $\mathcal{O}(V\,E^2)$.

— Bellman–Ford (par augmentation) : $\mathcal{O}(V\,E)$, multiplié par le nombre d'augmentations.

— Dijkstra + potentiels (par augmentation) : $\mathcal{O}(E \log V)$, multiplié par le nombre d'augmentations.

6 Validation et résultats

6.1 Jeux de test

Des fichiers d'exemples sont fournis (format « liste d'arcs »). En particulier :

— un exemple `maxflow` (coûts à 0) pour valider le flot maximum et la min-cut,

— un exemple `mincost` avec F fixé pour valider le flot de coût minimum.

6.2 Bench de cohérence

Un mode `bench` compare automatiquement les résultats (flow, cost) de Bellman–Ford et Dijkstra+potentiels pour plusieurs valeurs de F (jusqu'à $\min(F_{max}, 20)$). L'égalité des résultats constitue une vérification forte de la correction.

6.3 Exemple de sortie (résumé)

```
TOTAL_FLOW = 15
MIN_CUT_S ...
TOTAL_FLOW = 4 (target F=4)
TOTAL_COST = 12
NEGATIVE_CYCLE_IN_RESIDUAL = false
```

7 Conclusion

Le projet fournit une implémentation propre et reproductible des algorithmes demandés, avec une structure de données résiduelle unique réutilisée pour le flot maximum, la min-cut, et le flot de coût minimum (deux variantes). La validation par bench BF vs Dijkstra renforce la confiance dans la correction.